

Track 12 — Abrechnung und Finanzflüsse

Datum: 28.08.2026

Ausgangs-HEAD: b99a893 (Track 11, Betroffenenrechte und Löschkette)

Supabase-Projekt: nnwytkqibdxgimjyuq — ausschließlich lesend befragt

Prüfumfang: jeder produktive Codepfad, der Geld, Preise, Stundensätze, Rechnungen, Abrechnungszeiträume oder die Kassenintegration berührt

0. Angriffsfläche dieses Tracks

Die Tracks 1–11 haben nacheinander die Frage „**wer darf welche Daten sehen und schreiben**“ geschlossen: Mandantengrenze (6/7), Objektbindung (10), Rollenmatrix (9), Betroffenenrechte (11). Track 12 stellt eine andere Frage, die von keiner dieser Prüfungen beantwortet wird:

> Stimmt der Betrag?

Ein Zugriffsaudit kann diese Frage prinzipiell nicht beantworten. Wer eine Zeile schreiben **darf**, kann sie mit einem falschen Wert schreiben, und der Weg sieht in jedem Zugriffstest korrekt aus. Die interessanten Befunde dieses Tracks liegen deshalb sämtlich hinter einer bestandenen Berechtigungsprüfung: eine Pflegekraft, die ihre eigene Zeile bearbeitet; eine Server Action, die ihr eigenes Konto anlegt; ein Cron, der zweimal läuft; ein Trigger, der die richtige Tabelle liest und die falsche Zeile zieht.

1. Methode

Drei Quellen, in dieser Reihenfolge:

1. Ein nur lesendes Orakel gegen die Produktionsdatenbank.
`public._run_sql` mit `RAISE EXCEPTION` — die Transaktion rollt immer zurück, es kann per Konstruktion nichts schreiben und kein DDL ausführen. Gelesen wurden: effektive Rechte (`has_table_privilege`, `has_column_privilege`), sämtliche RLS-Policies der Geldtabellen, die Quelltexte von 12 Trigger-Funktionen und der Rechnungs-RPC, die Constraints, die generierten Spalten und die Verteilung der Nachweiszustände.
2. Ein Zugriffstest mit dem öffentlichen Schlüssel gegen neun Geldtabellen — die einzige Prüfung dieses Tracks, die von außen erfolgt.
3. Quelltextprüfung der 189 geldberührenden Routen und der Abrechnungsmodule.

Eine Korrektur an der eigenen Methode, die festgehalten gehört: die erste Rechteabfrage lief über `information_schema.role_table_grants` und meldete für `authenticated` null Rechte auf allen Geldtabellen. Das wäre ein spektakulärer Negativbefund gewesen — und war falsch. Diese Sicht zeigt nur Rechte, für die die abfragende Rolle selbst Grantor oder Grantee ist. Erst `has_table_privilege()` löst PUBLIC-Grants und Vererbung auf und zeigt die Wirklichkeit: `authenticated` hat auf ***acht von acht*** Geldtabellen `SELECT`, `INSERT` und `UPDATE`. Ein Audit, das an dieser Stelle nicht nachfragt, meldet die gesamte Angriffsfläche als geschlossen.

2. Befunde

B1 — Die Registrierung war das offene Gegenstück zur Stundensatz-Sperre

Schwere	P1
Ort	app/engel/register/actions.ts
Status	IMPLEMENTIERT · GETESTET · CI-GRÜN

Track 9 hat den Stundensatz an der Datenbank verriegelt. Das ist live belegt und gilt:

```
has_column_privilege('authenticated','public.angels','hourly_rate','UPDATE') = false
has_column_privilege('authenticated','public.angels','qualification','UPDATE') = false
has_column_privilege('authenticated','public.angels','is_certified','UPDATE') = false
has_column_privilege('authenticated','public.angels','is_45b_capable','UPDATE') = false
has_column_privilege('authenticated','public.angels','is_online','UPDATE') = true
has_table_privilege('authenticated','public.angels','UPDATE') = false
```

Im selben Commit wanderte die Registrierung auf den Admin-Client, der genau diese Sperre umgeht — und der Aufruf war ein upsert auf id:

```
const admin = createAdminClient()
await admin.from('angels').upsert({
  id: userId,
  hourly_rate: data.hourlyRate, // ← aus dem Aufruf
  qualification: data.qualification || null,
  is_certified: (data.qualification || '').includes('45b') || ...,
  is_45b_capable: (data.qualification || '').includes('45b'),
  total_jobs: 0, rating: 5.0, satisfaction_pct: 100,
})
```

`requireAuth()` in derselben Datei prüft ausschließlich, dass jemand angemeldet ist — keine Rolle, und vor allem nicht, ob dieses Konto bereits registriert ist. Ein upsert auf id ist idempotent per Konstruktion. Ein längst registrierter Engel konnte die Server Action ein zweites Mal aufrufen und dabei **genau die vier Spalten frei setzen, die Track 9 an der Datenbank verriegelt hatte**.

Server Actions sind aufrufbare HTTP-Endpunkte. Dass die Oberfläche das Feld gar nicht anbietet — `app/engel/register/page.tsx` schickt die Serverkonstante `ENGEL_HOURLY_RATE` — ist keine Schranke, sondern der Beleg dafür, dass der Wert nie aus dem Aufruf hätte kommen dürfen.

Drei Nebenwirkungen desselben upsert:

- `is_45b_capable` aus Freitext. Das Kennzeichen entstand aus `qualification.includes('45b')` — wer „45b“ in ein Freitextfeld tippt, trägt im Kundenportal das §45b-Abzeichen.
- `hourly_rate` ohne Obergrenze. Die einzige Prüfung war `> 0`.
- Bewertungshistorie löschar. Jeder erneute Aufruf stempelte `rating = 5.0`, `total_jobs = 0`, `satisfaction_pct = 100` zurück.

Abhilfe. Der Stundensatz kommt nicht mehr aus dem Aufruf, sondern aus `ENGEL_HOURLY_RATE`; der Parameter bleibt in der Signatur (Aufrufer brechen nicht) und wird ignoriert. Der Bestand wird gelesen: existiert die Zeile, werden nur noch die vier selbstgepflegten Felder fortgeschrieben (`services`, `availability`, `is_online` — dieselben, die der Spalten-GRANT

erlaubt); Stundensatz, Qualifikation, Kennzeichen und Bewertung bleiben stehen. Fail-closed: ist der Bestand nicht lesbar, wird gar nicht geschrieben — sonst wäre der Befund über einen provozierten Lesefehler wieder erreichbar.

Einordnung, die dazugehört: angels.hourly_rate fließt heute in keinen Rechnungsweg. Der Rechnungslauf zieht seine Preise ausschließlich aus billing_tariffs (siehe N1). Der Befund ist deshalb P1, nicht P0 — er ist eine freie Änderung an Vergütungs- und Qualifikationsdaten, kein direkter Griff in eine Rechnung.

B2 — „Unterschrieben“ ohne Unterschrift, an der Datenbank vorbei

Schwere	P1
Ort	Policy sr_engel_own · compute_signature_hash · create_invoice_draft_atomic
Status	Anwendungsseite IMPLEMENTIERT · GETESTET · CI-GRÜN — Datenbankseite eingecheckt, NICHT angewendet (20261017000000)

Die Unterschriftspflicht wird bisher nur auf der schreibenden Seite durchgesetzt: assertKlientenUnterschrift verhindert, dass /api/leistungsnachweis/crud den proof_status ohne mitgeschickte Unterschrift auf UNTERSCHRIEBEN setzt. Das ist richtig, und es deckt genau einen Weg ab.

Daneben liegt ein zweiter Weg, der diese Route nicht benutzt. Live gilt:

```
has_table_privilege('authenticated','public.service_records','UPDATE') = true (ohne Spalteneinschränkung)
```

```
pg_policies (service_records):
sr_engel_own | ALL | PERMISSIVE | {authenticated}
USING/CHECK = caregiver_id IN (SELECT eigene_caregiver_ids()) OR is_admin()
service_records_caregiver_update | UPDATE | PERMISSIVE | {public}
USING = caregiver_id IN (...) AND status = ANY (ARRAY['draft','incomplete'])
```

Permissive Policies werden ODER-verknüpft. Die eng gefasste service_records_caregiver_update hat deshalb keine einschränkende Wirkung mehr — sr_engel_own lässt denselben Schreibvorgang in jedem Status durch. Die Statusbeschränkung, die jemand bewusst hingeschrieben hat, ist tot.

Eine Pflegekraft kann damit auf ihrer eigenen Zeile:

```
PATCH /rest/v1/service_records?id=eq.<eigene Zeile>
{ "proof_status": "UNTERSCHRIEBEN" }
```

Was danach passiert, steht in den Trigger-Quelltexten (live gelesen):

1. sync_service_record_status (BEFORE) hebt status auf 'signed' — der Nachweis gilt als abrechenbar.
2. compute_signature_hash (BEFORE) läuft nicht: die Funktion verlangt proof_status = 'UNTERSCHRIEBEN' AND client_signed_at IS NOT NULL. Ohne Zeitstempel bleibt signature_hash NULL — und **is_locked bleibt FALSE**, der Nachweis also weiter veränderbar.
3. create_invoice_draft_atomic zählt einen Nachweis nur dann als unsigniert, wenn

sr.proof_status IS DISTINCT FROM 'UNTERSCHRIEBEN' AND sr.signature_hash IS NULL

— eine ODER-Annahme: der bloße Statuswert genügt der Sperre.

Die Kette schließt sich über POST /api/billing/auto-invoice. Dessen authorize() lässt ausdrücklich auch die Pflegekraft mit Native-Bearer-Token zu und schreibt danach mit dem Dienstschlüssel. ****Ein einzelnes Pflegekraft-Konto kann vom Nachweis bis zur fertigen Rechnung durchlaufen, ohne dass je ein Kunde unterschrieben hat.****

Abhilfe, zweistufig.

Anwendungsseite (wirkt sofort): lib/billing/nachweis-beleg.ts stellt die Frage auf der lesenden Seite noch einmal, und strenger: nicht „steht da UNTERSCHRIEBEN“, sondern „gibt es einen Beleg“ — Hash zusammen mit Zeitstempel, oder ein Unterschriftsbild, oder eine Zeile in service_signatures. assertBelegteNachweise sitzt in createInvoiceDraft, also an der einen Stelle, durch die jeder Rechnungsweg läuft (Einzelrechnung, auto-invoice, Sammelrechnungslauf). Die Auswahl bildet die WHERE-Klausel der RPC exakt nach — ein Test prüft genau das, denn ein Guard, der eine andere Menge liest als die abgerechnete, ist wertlos, auch wenn er grün meldet.

Datenbankseite (Migration 20261017000000, eingecheckt, **nicht angewendet**): ein BEFORE-Trigger enforce_unterschrift_beleg, der die Frage an der Quelle stellt statt am Tor, und das Entfernen der FOR-ALL-Policy sr_engel_own. Die drei richtig gefassten Policies darunter (..._caregiver_read, ..._caregiver_insert, ..._caregiver_update) decken ab, was die Pflegekraft braucht.

Bewusst nicht über signature_hash geprüft: den setzt compute_signature_hash selbst, und welcher von zwei BEFORE-Triggern zuerst läuft, entscheidet die alphabetische Reihenfolge ihrer Namen. Eine Prüfung darauf wäre von einer Umbenennung abhängig.

B3 — Der PfluV-Deckel greift, und er greift falsch

Schwere	P2 · LIVE_VERIFIZIERT
Ort	Trigger enforce_tariff_obergrenze
Status	Warnung IMPLEMENTIERT · GETESTET · CI-GRÜN — Trigger-Fix eingecheckt, NICHT angewendet (20261017000002)

billing_gesetzliche_obergrenzen trägt zwei hessische §45b-Zeilen, die sich ausschließlich im anbotstyp unterscheiden:

bundesland	rechtsgrundlage	angebotstyp	leistungsart	vergütungsart	cent	bestätigt	aktiv
hessen	§45b SGB XI	betreuungsangebot	NULL	zeit_stunde	3000	true	true
hessen	§45b SGB XI	entlastungsangebot	NULL	zeit_stunde	2500	true	true

lib/billing/obergrenzen.ts dokumentierte, der Seed stehe „bewusst auf FALSE“ und der Trigger greife deshalb nicht. ****Das gilt live nicht mehr:**

beide Zeilen stehen auf bestaetigt = TRUE.** Der Trigger greift.

Und weil er den anbotstyp nicht kennt — billing_tariffs hat die Spalte gar nicht —, sind beide Zeilen für seinen Filter gleichwertig; sein ORDER BY bricht den Gleichstand nicht auf, und LIMIT 1 entscheidet der Planer. Die Auswahl wurde gegen die Produktion nachgestellt:

Leistungsart	Soll nach §45a Abs. 1 S. 2 SGB XI	Trigger wählt	
betreuung_45a	3000 (Nr. 1)	3000	richtig
demenzbetreuung	3000 (Nr. 1)	3000	richtig
hauswirtschaft	2500 (Nr. 3)	3000	zu hoch
einkaufsservice	2500 (Nr. 3)	3000	zu hoch

Der Befund wirkt in beide Richtungen: Entlastungsleistungen dürfen 20 % teurer sein, als die Verordnung erlaubt — und zöge der Planer die andere Zeile, würde umgekehrt ein rechtmäßiger 30-EUR-Betreuungstarif abgewiesen. **Eine Sperre auf einem Gleichstand ist keine verlässliche Sperre.**

Die Auswahl des Anwendungscodes ist die richtige: er kennt die Zuordnung (ANGEBOTSTYP_VON_LEISTUNGSART) und filtert danach. Neu ist abweichungZurDatenbank(), die beide Auswahlen nebeneinanderstellt und den Unterschied benennt — die Tarifpflege bekam sonst entweder eine Sperre ohne Grund oder ein stillschweigendes Durchwinken über der gesetzlichen Grenze. Die Route POST /api/billing/tariffs gibt die Meldung mit aus.

B4 — Der Unschärfe-Hinweis fiel genau dann weg, wenn er zählt

Schwere	P3
Ort	lib/billing/obergrenzen.ts::pruefeGegenRegeln
Status	IMPLEMENTIERT · GETESTET · CI-GRÜN

Für Leistungsarten, deren Angebotstyp nach §45a nicht eindeutig ist (alltagsbegleitung, begleitservice, sonstige), prüft das Modul gegen die höchste einschlägige Grenze und weist die Unschärfe in der Meldung aus. Dieser Hinweis stand nur im Warn-Zweig. Solange alle Regeln unbestätigt waren, war das dasselbe. Seit die Zeilen live auf bestaetigt = TRUE stehen, läuft der häufigere Fall in den Verstoß-Zweig — und dort fehlte der Hinweis genau dann, wenn er am meisten zählt: die Meldung sagte „die Datenbank weist das zurück“, ohne zu sagen, dass die Grenze unter Unschärfe gewählt wurde. Jetzt steht er in beiden Zweigen.

Der Befund fiel auf, weil ein Test ihn erwartet hatte und rot wurde — er war die Korrektur wert, nicht der Test.

B5 — Ein Nachtdienst erzeugt eine Rechnungsposition, die Geld abzieht

Schwere	P1 (latent — 0 betroffene Zeilen live)
Ort	service_records.duration_minutes · alle vier Erfassungswege
Status	Anwendungsseite IMPLEMENTIERT · GETESTET · CI-GRÜN — CHECK eingecheckt, NICHT angewendet (20261017000000)

duration_minutes ist live eine GENERATED-Spalte:

```
duration_minutes GENERATED ALWAYS AS ((EXTRACT(epoch FROM (end_time - start_time))::integer / 60) STORED
```

Und genau dieser Wert bestimmt den Rechnungsbetrag:

```
WHEN 'zeit_stunde' THEN ROUND((preis_cent::NUMERIC / 100.0) * (COALESCE(duration_minutes, 60)::NUMERIC / 60.0), 2)
```

Es gibt live weder einen CHECK-Constraint noch eine Prüfung im Anwendungscode, die end_time > start_time verlangt. start_time und end_time sind time without time zone. Ein Nachtdienst von 22:00 bis 06:00 ergibt

```
duration_minutes = (6·60) – (22·60) = –960
```

und damit eine Rechnungsposition mit negativem Betrag: sie zieht Geld ab, statt es zu fordern. Der Einsatz wird nirgends abgelehnt und nirgends gemeldet.

Dass dieser Fall vorgesehen ist und nicht exotisch, steht in derselben Datenbank: billing_tariffs führt nacht_von, nacht_bis und zuschlag_nacht_prozent, und Track 8 hat die Doppelbelegungsprüfung ausdrücklich über Tagesgrenzen inklusive Nachtdienst geprüft.

Dass die richtige Antwort im Repo bekannt ist, steht ebenfalls in der Datenbank: **angel_availability trägt den CHECK angel_availability_zeitfenster_gueltig.** Für service_records und assignments fehlt das Gegenstück.

Abhilfe. lib/leistungsnachweis/zeitraum.ts bildet die Rechnung der generierten Spalte exakt nach und weist drei Fälle ab: Ende vor Beginn, Ende gleich Beginn (Position über 0,00 EUR) und unlesbare Zeiten (fail-closed — ein stillschweigendes „dann eben ohne Prüfung“ wäre hier der gefährlichere Ausgang, denn die generierte Spalte rechnet trotzdem). Angewandt an allen Erfassungswegen: /api/leistungsnachweis/crud, /api/billing/sgb-v/leistungsnachweise und PATCH /api/tours/[id]/stops (dort im fail-soft-Stil der Umgebung, als serviceRecordFehler).

Bewusst nicht getan: bei end < start 24 Stunden aufzuschlagen und den Nachtdienst korrekt zu berechnen. Die abgerechnete Spalte ist generiert und rechnet ohne diesen Zuschlag weiter — die Anwendung würde dann eine andere Dauer melden als die, die in die Rechnung geht. Solange die Spalte so rechnet, wie sie rechnet, ist die einzige ehrliche Antwort: den Einsatz über Mitternacht in zwei Nachweise teilen und das auch so sagen.

Bestandslage: live 0 Zeilen mit end_time < start_time, 0 mit NULL-Zeiten. Der Befund ist latent, der CHECK ließe sich ohne Bereinigung anlegen.

B6 — Die Mahngebühr ist ein Lesen-Rechnen-Schreiben ohne Vergleich

Schwere	P2
Ort	lib/billing/core/dunning.ts::advanceDunning
Status	IMPLEMENTIERT · GETESTET · CI-GRÜN

Die Eskalation las den Mahneintrag, rechnete die nächste Stufe und die Gebühr aus und schrieb dann:

```
.update({
  dunning_level: newLevel,
  dunning_fee_cents: (entry.dunning_fee_cents || 0) + feeCents, // ← aufaddiert
...
})
.eq('invoice_id', invoiceId)
.eq('organization_id', organizationId)
// kein Vergleich mit dem gelesenen Stand
```

Die Stufe ist dabei idempotent — zweimal derselbe Wert. Die Gebühr ist es nicht: sie wird aufaddiert. Laufen zwei Eskalationen desselben Eintrags nebeneinander — der tägliche Cron /api/cron/mahnlauf und ein manueller Anstoß über /api/billing/dunning/advance oder /lauf, oder schlicht ein Wiederholungslauf desselben Vercel-Crons —, dann lesen beide denselben Stand und buchen die Mahngebühr zweimal. Der Kunde bekommt eine Gebühr in Rechnung gestellt, die es nur einmal gibt.

Abhilfe. Compare-and-Swap: der gelesene Stand steht jetzt als .eq('dunning_level', entry.dunning_level) im Schreibvorgang, und das Ergebnis wird ausgewertet (.select('id') — ohne das meldet PostgREST keinen Fehler, wenn null Zeilen getroffen wurden). Null Treffer werfen MahnstufeBereitsEskaliertError; runDunningRun verbucht den als unverändert statt als Fehler, und — das eigentlich Wichtige — der Eintrag landet nicht in result.eskaliert und löst damit keinen zweiten Mahnungsversand aus.

Der Testdoppelgänger in __tests__/billing/mahnlauf.test.ts gab bei update bisher immer data: null zurück. Ein CAS ließ sich damit nicht prüfen — er sähe aus wie ein Fehlschlag. Er liefert jetzt wie PostgREST die betroffenen Zeilen.

B7 — Der Manipulationsschutz hat in dieser Datenbank noch nie gegriffen

Schwere	P2 · LIVE_VERIFIZIERT · Bestandsbefund
Ort	service_records (Produktionsdaten)
Status	BENANNT — keine Datenänderung vorgenommen

Verteilung aller 30 Zeilen, live am 28.08.2026 gelesen:

status	proof_status	hash	signed_at	Unterschrift	locked	n
draft	ENTWURF	-	-	ja	nein	2
signed	ENTWURF	-	-	ja	nein	10

signed	ENTWURF	-	-	nein	nein	3
invoiced	ENTWURF	-	-	ja	nein	14
invoiced	ENTWURF	-	-	nein	nein	1

Daraus folgen drei Tatsachen:

1. ****signature_hash** und **client_signed_at** sind auf KEINER der 30 Zeilen gesetzt. ****compute_signature_hash** hat in dieser Datenbank nie ausgelöst.
2. ****is_locked** ist auf allen 30 Zeilen FALSE — auch auf den 15 bereits abgerechneten. ****Der Trigger prevent_locked_record_change**, der ausdrücklich als „Manipulationsschutz aktiv“ auftritt, schützt derzeit nichts. Er hängt vollständig daran, dass **compute_signature_hash** vorher gelaufen ist.
3. Vier Nachweise tragen überhaupt keinen Unterschriftsnachweis — drei davon signed, einer bereits invoiced.

Ergänzend: weil **proof_status** auf allen Zeilen ENTWURF ist, würde **create_invoice_draft_atomic** heute für jeden dieser Nachweise mit **MISSING_SIGNATURE** abbrechen. Die 15 invoiced-Zeilen stammen also aus der Zeit vor dieser Sperre (RPC v8, Migration 20260911010000). Das ist die andere Seite desselben Befundes: die Sperre wirkt nach vorn, der Bestand darunter ist nicht von ihr gedeckt.

Bewusst nicht behoben. Ob diese vier Nachweise nachträglich zu unterschreiben, zu stornieren oder als Altbestand zu belassen sind, ist eine fachliche und aufbewahrungsrechtliche Entscheidung (§ 630f BGB) — sie wird hier nicht getroffen und nicht durch einen Backfill vorweggenommen. **npm run verify:abrechnung** meldet die Zahl bei jedem Lauf.

3. Negativbefunde — was ausdrücklich NICHT gefunden wurde

Ein Audit muss auch sagen, was es geprüft und für in Ordnung befunden hat. Sonst liest sich jede spätere Änderung an diesen Stellen wie ein ungeprüftes Risiko.

N1 — Rechnungsbeträge kommen nicht aus dem Client. (**LIVE_VERIFIZIERT** am Quelltext der RPC.) **create_invoice_draft_atomic** löst je Nachweis den Tarif aus **billing_tariffs** auf und rechnet **preis_cent × duration_minutes** plus Zuschläge. **service_records.amount** — das einzige Betragsfeld, das ein Client setzen kann — geht nicht in den Betrag ein; es wird nur als **abweichung_cent/abweichung_grund** mitprotokolliert. Der naheliegendste Angriff „Betrag im Request-Rumpf“ existiert an dieser Stelle nicht.

N2 — 131 €, nicht 125 €. **ENTLASTUNG_MONATLICH_EUR** = 131, **ENTLASTUNG_JAEHRLICH_EUR** = 1572, **VP_KZP_KOMBINIERT_EUR** = 3539. Die Werte 125/1500 stehen ausschließlich im historischen 2024-Eintrag (damit Rechnungen von 2024 reproduzierbar bleiben) und in Blog-Texten, die sie ausdrücklich als überholt kennzeichnen. Live trägt **client_budgets** in allen 4 Zeilen **annual_amount** = 1572.00.

N3 — **duration_minutes** ist nicht schreibbar. Die Spalte ist **GENERATED ALWAYS**; ein mitgelieferter Wert lässt den **INSERT** mit 428C9 scheitern. Und sie leitet sich aus genau den beiden Feldern ab, die der Signatur-Hash abdeckt (**start_time**, **end_time**) — die Unterschrift

attestiert also dieselben Tatsachen, aus denen der Betrag entsteht.

N4 — Die Track-9-Sperre ist live. Nicht die Migrationsdatei zählt, sondern `has_column_privilege` (siehe B1). Eine ältere Notiz, die Migration 20261015000000 sei „noch nicht live“, ist damit widerlegt.

N5 — anon kommt an keine Geldtabelle. Neun Tabellen mit dem öffentlichen Schlüssel abgefragt, alle HTTP 401.

N6 — Alle neun Crons sind fail-closed geschützt. Jeder ruft `pruefeCronGeheimnis` aus `lib/api/cron-auth.ts`.

N7 — Rechnungen sind nach Festschreibung unveränderlich. Zwei unabhängige Trigger: `prevent_finalized_invoice_mutation` friert bei 17 Statuswerten Betrag, Zeitraum, Kunde, Kostenträger und Rechnungsnummer ein; `validate_invoice_status_transition` friert zusätzlich alles ab `frozen_at` IS NOT NULL und erzwingt eine vollständige Statusübergangsmatrix.

N8 — Der Rechnungslauf ist idempotent. ``idempotency_key = inv_<client>_<monat>_<budgettyp>_v4``; ein zweiter Aufruf gibt die bestehende Rechnung zurück, statt eine zweite anzulegen. Live läuft `v10_storno_ausschluss`.

N9 — Der Budgetdeckel ist verdrahtet. `wendeBudgetDeckelAn` wird in `createInvoiceDraft` aufgerufen; §45b wird zusätzlich zum Jahresdeckel kumuliert monatlich gedeckelt ($131 \text{ €} \times \text{Monatsindex} + \text{Übertrag}$), der Überschuss wandert auf den Privatanteil statt blockiert zu werden. §36 bleibt ausdrücklich ungedeckelt und begründet (UNGEDECKELTE_TOEPFE).

N10 — Doppelte Zuordnung ist an der Datenbank ausgeschlossen. `payment_allocations_payment_id_invoice_id` key UNIQUE; dazu `idx_service_records_unique_entry` UNIQUE auf `(caregiver_id, client_id, date, start_time)` WHERE status <> 'incomplete'.

N11 — §45b ist praktisch nicht abrechenbar. Von 23 Tarifen sind 12 auf blocked (alle §45b- und §39-Zeitstundentarife zu 35 €/h — über der PfluV-Grenze), 11 auf verified (10 privat, 1 Wegepauschale). Die RPC verlangt für Nicht-Privat `tarif_status = 'verified'`. Der Deckel wirkt hier also, wenn auch über den Umweg der Tarifsperre.

N12 — `pruefeMahnbarkeit` prüft alle zehn Sperren. Die Eskalation kommt an ihm nicht vorbei, auch kein künftiger Aufrufer.

4. Restposten — benannt und NICHT behoben

R1 — Ein verifizierter §45b-Tarif ohne benannte Rechtsgrundlage. `wegepauschale / §45b SGB XI / wegepauschale / 500 Cent` steht live auf `ist_aktiv = true`, `tarif_status = 'verified'` und ist damit der einzige §45b-Tarif, über den heute eine Kassenrechnung entstehen könnte. `lib/billing/obergrenzen.ts` hält zu genau dieser Position fest, die 5-EUR-Pauschale sei „laut Quellenprüfung ohne PfluV-Grundlage und deshalb auch nicht geseedet“. Zwei Stellen im System sagen Gegenteiliges. Welche recht hat, ist eine rechtliche Frage und wird hier nicht entschieden.

R2 — Die Zugangssperre für anon hängt an einem Funktionsrecht, nicht an einer Policy. Die 401 aus N5 kommen bei fünf der neun Tabellen als `permission denied for function current_org_id` — nicht aus einer RLS-Entscheidung. `invoices` und `invoice_items` tragen zusätzlich eine `RESTRICTIVE ..._anon_deny`-Policy; `client_budgets`, `service_records`,

payments, billing_tariffs und leistungspreise tragen keine. Wer EXECUTE ON current_org_id an anon zurückgibt, öffnet diese fünf Tabellen in einem Zug. Das ist heute keine Lücke, aber eine Verteidigungslinie weniger, als es aussieht.

****R3** — assignments hat denselben Zeitfenster-Befund wie service_records.** Auch dort fehlt der CHECK end_time > start_time (live: 0 von 5 Zeilen betroffen). Migration 20261017000000 fasst assignments bewusst nicht an: dort sind Nachtdienste über Mitternacht laut Track 8 ein regulär unterstützter Fall, und ein CHECK würde sie verbieten. Die richtige Antwort dort ist eine andere als hier — sie gehört in einen eigenen Vorgang.

R4 — Der Budgetdeckel ist nach vorn offen. Der 2025er-Eintrag trägt gueltigBis: '9999-12-31'. Ab dem 01.01.2028 — der nächsten Dynamisierung nach § 30 SGB XI — rechnet das Modul still mit den Sätzen von 2025 weiter, statt zu werfen. Die Fail-closed-Zusicherung gilt nur rückwärts. Das ist eine bewusste Entscheidung (die Alternative wäre, an einem Stichtag die gesamte Abrechnung anzuhalten) und als Test festgehalten, damit sie nicht für eine Zusicherung gehalten wird. Ein Satz für 2028 wird hier nicht erfunden.

5. Migrationen — eing_checked, NICHT angewendet

DDL über den Dienstschlüssel wird live mit 42501 abgewiesen; beide Migrationen warten auf die manuelle Ausführung im SQL-Editor. Zu jeder gibt es einen Rollback.

Datei	Wirkung
20261017000000_abrechnungsintegritaet_leistungsnachweis.sql	Trigger enforce_unterschrift_beleg (B2) · CHECK service_records_zeitfenster_gueltig (B5) · entfernt die FOR-ALL-Policy sr_engel_own (B2)
20261017000001_rollback_...	stellt sr_engel_own mit exakt dem live gelesenen Ausdruck wieder her
20261017000002_obergrenze_angebotstyp.sql	Funktion anbotstyp_von_leistungsart · enforce_tariff_obergrenze filtert nach Angebotstyp und löst den Gleichstand über die höchste Grenze auf (B3)
20261017000003_rollback_...	stellt den Trigger in der live gelesenen Fassung wieder her

Verletzt eine der beiden heute Bestandsdaten? Nein. Live: 0 Zeilen mit end_time < start_time, 0 Zeilen mit `proof\_status` IN ('UNTERSCHRIEBEN','ABGERECHNET')`. Beide lassen sich ohne Bereinigung anwenden.

6. Prüfstand

Neue Tests: 45, davon 18 Gegenproben

Zu jedem Befund gehört ein Paar: eine Gegenprobe, die den alten Zustand nachstellt und verlangt, dass er jetzt scheitert — und ein Nachweis, dass derselbe Vorgang ohne die Manipulation weiterhin

durchläuft. Ohne das zweite wäre „alles gesperrt“ ebenfalls grün, und die Sperre kein Beweis.

Datei	Tests	Deckt ab
__tests__/track12-abrechnung-finanzfl uesse.test.ts	35	B2, B3, B4, B5, N2, R4
__tests__/track12-mahnstufe-cas.test. ts	4	B6
__tests__/track12-engel-registrierung. test.ts	6	B1

Ausgewählte Gegenproben:

- 22:00–06:00 ergibt –960 Minuten und wird abgewiesen — und 06:00–22:00 läuft unverändert durch.
- `proof_status='UNTERSCHRIEBEN'` allein ist kein Beleg — und ein Nachweis mit Hash + Zeitstempel bzw. Unterschriftsbild kommt durch.
- Ein Hash ohne `client_signed_at` zählt nicht: so einen Hash bildet `compute_signature_hash` nie.
- Ein mitgeschickter Stundensatz von 999 landet nicht in der Datenbank — und die normale Registrierung legt die Zeile weiterhin an.
- Der zweite Registrierungsaufwurf fasst die vier gesperrten Spalten nicht mehr an und setzt Bewertung/Einsatzzahl nicht zurück.
- Der DB-Trigger kann die beiden Obergrenzen-Zeilen nicht auseinanderhalten (gleichwertig = 2) — und ein rechtmäßiger 30-EUR-Betreuungstarif bleibt „eingehalten“.
- Trifft der Mahn-CAS nicht, wird weder eine Gebühr gebucht noch die Rechnung nachgezogen.
- `assertBelegteNachweise` bildet die WHERE-Klausel der RPC nach (Org-Fence, `budget_type`, `status` IN ('signed','complete'), Monatsgrenzen inklusive Schaltjahr) — ein Guard, der eine andere Menge liest als die abgerechnete, ist wertlos, auch wenn er grün meldet.
- Der Guard schreibt nichts: ein Guard, der schreibt, ist kein Guard.

Sechs Bestandstests an die neue Regel gezogen — die Regel nicht gelockert

- `track9-personalverwaltung-audit.test.ts`: zwei Prüfungen suchten wörtlich nach `from('angels').upsert`. Der `upsert` war die Umgehung. Die Zaunregel gilt jetzt für alle Schreibformen statt nur für den `upsert` — sie ist damit schärfer, nicht lockerer. Plus eine neue Prüfung, dass `hourly_rate` nicht mehr aus data kommt.
- `abrechnungskette-pglite.test.ts`, `sammelrechnung-e2e-phase4-pglite.test.ts`: vier Fixtures setzten `signature_hash` ohne `client_signed_at` — ein Zustand, den `compute_signature_hash` nie erzeugt. Zeitstempel ergänzt.
- `tariff-based-invoice.test.ts`, `billing-e2e.test.ts`: die Mocks kannten die neue `service_records`-Abfrage nicht.
- `mahnlauf.test.ts`: der Doppelgänger gab bei `update` immer `data: null` zurück — siehe B6.
- `pre-pilot-snapshot.test.ts`: JUENGSTE_MIGRATIONEN nachgezogen.

Läufe

Prüfung	Ergebnis
tsc --noEmit	0 Fehler

vitest run	7855 grün / 0 rot (vorher 7809)
npm run test:unit (node:test)	2513 grün / 0 rot
npm run lint:forbidden	0 Treffer (24.811 Dateien)
npm run lint:route-auth	0 Treffer (412 Routen)
npm run lint:org-id	0 Treffer (1.419 Dateien)
npm run verify:abrechnung (neu, live)	5 von 7 bestanden, 2 Berichte

Die zwei offenen Punkte des Live-Prüfskripts sind genau die beiden Migrationen, die auf ihre Anwendung warten (D1/E1 und F2/G2) — das Skript schreibt das in seiner Schlussmeldung selbst hin.

7. Was dieser Track ausdrücklich nicht getan hat

- Keine Rechnung, Mahnung oder Zahlung ausgelöst. Kein Versand-Flag angefasst, kein Vercel-Flag gesetzt.
- Keine produktiven Daten verändert. Sämtliche Datenbankzugriffe dieses Tracks liefen über ein Orakel, das seine Transaktion zurückrollt, oder als GET mit dem öffentlichen Schlüssel.
- Keine Preise erfunden. Weder ein 2028er-Budgetsatz (R4) noch eine Rechtsgrundlage für die Wegepauschale (R1).
- Der Angriff aus B2 wurde nicht ausgeführt. Er ist strukturell belegt — aus den live gelesenen Policies, Grants und Trigger-Quelltexten — und in Tests nachgestellt, nicht gegen die Produktion gefahren.